Introduction

These notes explain how Hadoop Distributed File System (HDFS) and MapReduce work in practice, focusing on *how data is stored, managed, and processed at massive scale*. The content walks step-by-step through HDFS architecture, block design, metadata management, write and read paths, rack awareness, and the MapReduce programming model. The aim is to build a clear mental model of what happens when large files are uploaded and processed in Hadoop.

HDFS Architecture: Core Roles

NameNode – The Metadata Master

The NameNode is the master server that manages the filesystem namespace and metadata.

Responsibilities

- Maintains directory structure and file names
- Tracks which files map to which blocks
- Knows exact DataNode locations of each block
- Enforces permissions and replication factors
- Logs all metadata changes

What It Does NOT Do

- Does not store actual file data
- Does not handle client data transfer

Why It Matters

- It is the single source of truth
- Losing it makes data blocks unusable (no index)

DataNodes – The Storage Workers

DataNodes store actual file blocks on local disks.

Responsibilities

- Store fixed-size blocks (default 128 MB)
- Maintain checksums for integrity
- Send heartbeats and block reports
- Perform replication when instructed

What They Don’t Know

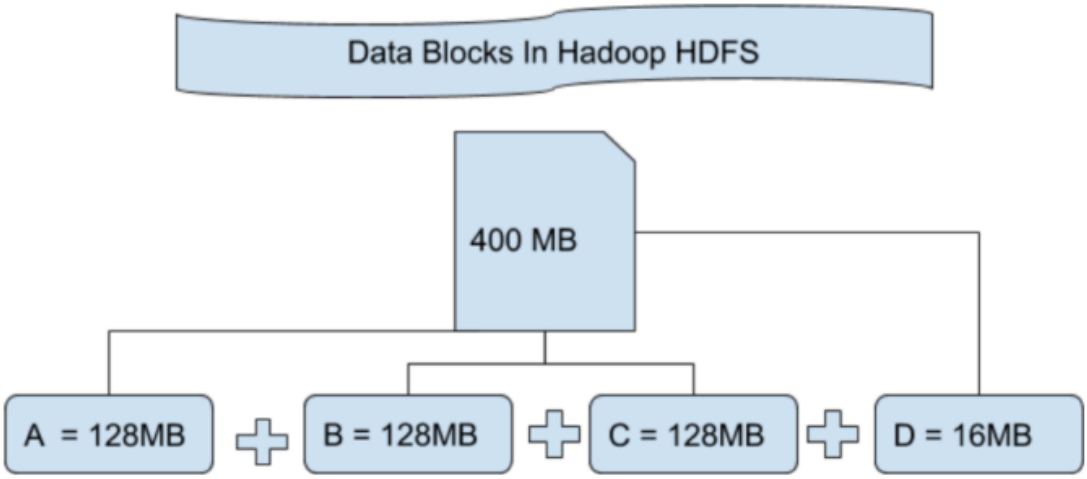
- File names
- File structure
- Full dataset context

Design Principle

Component	Focus
NameNode	Metadata only
DataNodes	Data only

Separation improves scalability and performance.

Giant Block Design



Why 128 MB Blocks?

Hard drives have two speeds:

Metric	Meaning
Seek Time	Time to position disk head
Transfer Time	Time to read/write data

Seek is slow (~10 ms), transfer is fast (~100 MB/s).

Vinyl Record Analogy

- Seek time = needle finding song
- Transfer time = music playing

Goal: minimize seeks, maximize transfer.

Performance Math

Scenario	Laptop (4 KB)	HDFS (128 MB)
Blocks for 1 GB	250,000	8
Seek Time	~41 min	0.08 sec
Transfer	10 sec	10 sec
Total	~41 min	~10 sec

HDFS prioritizes transfer efficiency.

Small File Problem

- A 1 KB file uses 1 KB disk
- But consumes one NameNode RAM entry
- Millions of tiny files overload memory
- Many small files cause many seeks

Rule

HDFS prefers large files and large streaming reads.

NameNode Metadata Internals

Daemon Concept

A daemon is a long-running background service that:

- Starts with the system
- Handles requests continuously
- Examples: NameNode, DataNode

FsImage

Snapshot of entire namespace.

Contains

- Directory tree
- File metadata
- Permissions
- Block mappings

Purpose

- Provides baseline metadata state
- Loaded into memory on startup

EditLog

Append-only log of changes.

Records

- Create
- Delete
- Rename
- Permission changes
- Replication updates

Function

- Replayed after FsImage load
- Brings metadata up to date

Checkpointing

Checkpoint merges:

FsImage + EditLog → New FsImage

Benefits

- Prevents massive logs
- Speeds restarts
- Keeps metadata manageable

Secondary NameNode

Role

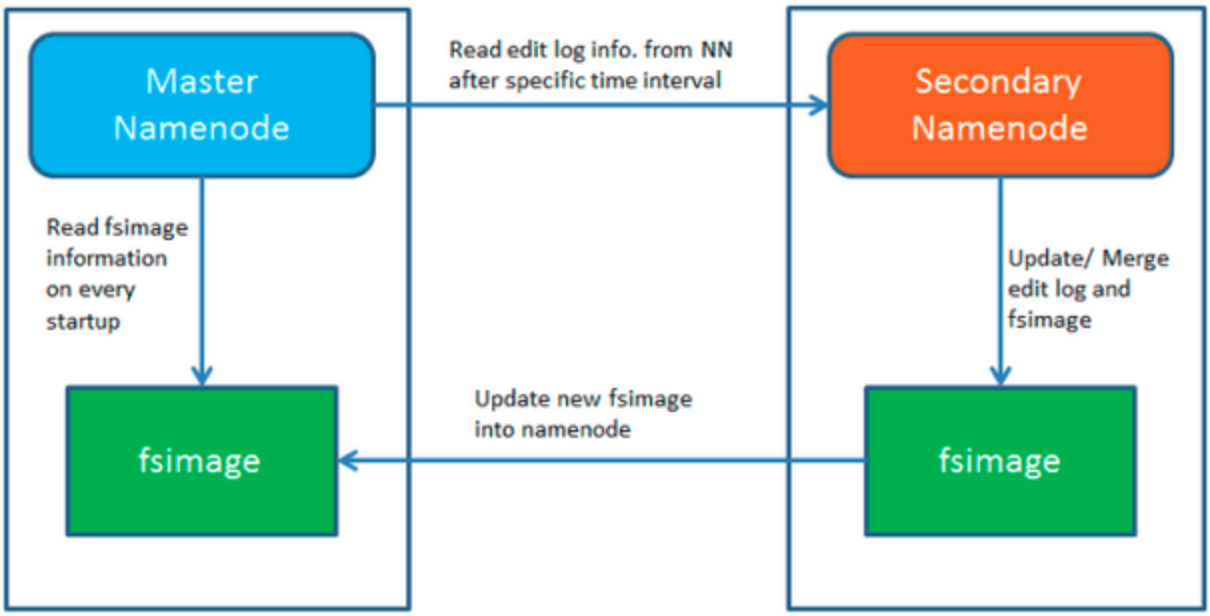
Checkpoint helper only.

Function

- Periodically merges FsImage and EditLog
- Sends updated snapshot back

Important Clarification

- NOT a backup
- Cannot replace failed NameNode



High Availability NameNodes

Modern Hadoop uses:

Node	Role
Active	Handles requests
Standby	Backup master
JournalNodes	Store shared logs

Mechanism

- Majority confirmation for edits
- Standby monitors updates
- Automatic failover

Removes single point of failure.

HDFS Write Pipeline

Step 1 – Client Request

Client asks NameNode to create file.

NameNode:

- Validates permissions
- Logs metadata change
- Provides block IDs
- Provides replica DataNodes

Step 2 – Pipeline Formation

Example chain:

DN-17 → DN-42 → DN-109

Step 3 – Packet Streaming

- Client sends 64 KB packets
- Each DataNode writes and forwards
- Last node sends ACK backward
- Client clears memory only after ACK

Benefit

- Efficient network use

- Simultaneous replication
- High durability

Failure Handling

If a node fails:

- ACKs stop
- Client informs NameNode
- New pipeline created
- Only affected block retried

Ensures uninterrupted uploads.

HDFS Read Path

Step 1 – Metadata Request

Client asks NameNode for block locations.

NameNode returns:

- Block IDs
- Replica locations
- Sorted by proximity

Step 2 – Direct Data Access

Client reads from nearest DataNode.

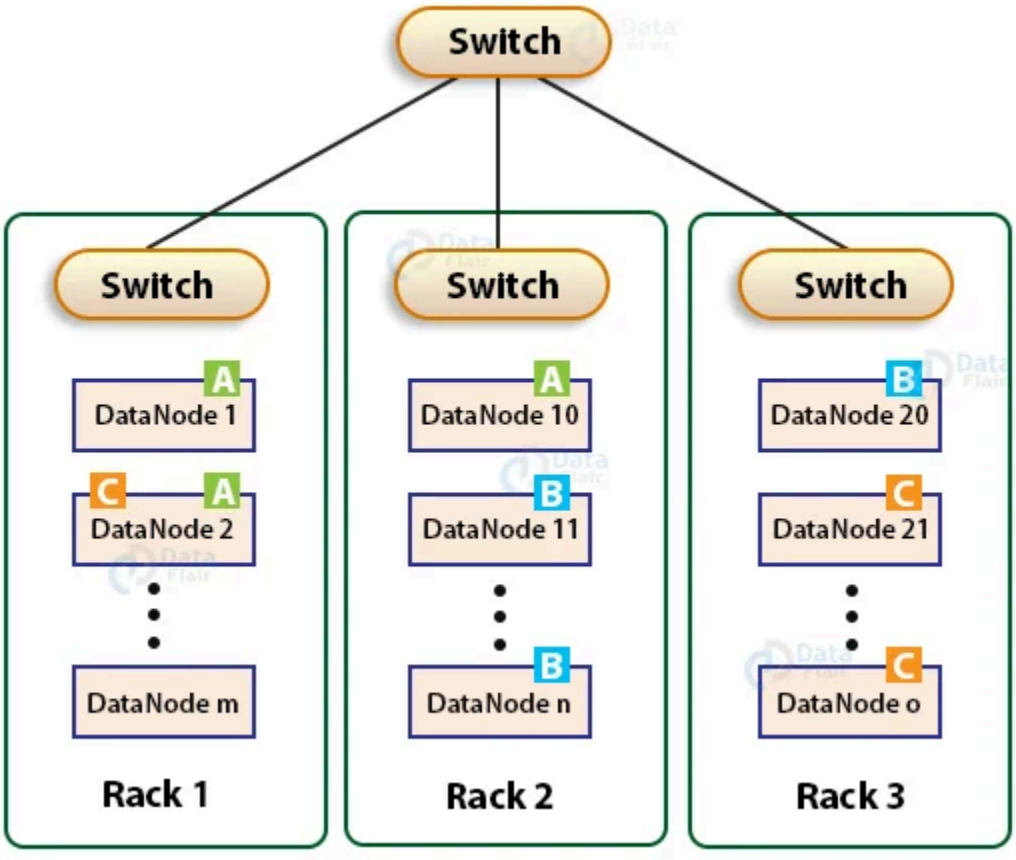
Data Locality Principle

Computation runs where data lives.

Advantages

- Reduced network usage
- Faster processing
- Better scalability

Rack Awareness



What is a Rack?

Group of nodes on same switch.

Shared:

- Power
- Network switch
- Failure risk

Replica Placement Policy

Replica	Location
1	Local node
2	Same rack
3	Different rack

Why Two in Same Rack?

Factor	Reason
Speed	Faster intra-rack bandwidth
Efficiency	Reduced inter-rack traffic
Fault Model	Node failures more common than rack failures

Survivability

System tolerates:

- Disk failure
- Server failure
- Rack failure

Topology known via admin scripts.

MapReduce: Distributed Compute Model

Purpose

Process massive datasets in parallel.

Core Ideas

From functional programming:

Function	Meaning
Map	Apply function to each item
Reduce	Combine results

Google's Additions

Feature	Benefit
Data locality	Minimal network use
Auto parallelism	Programmer simplicity
Re-execution	Fault tolerance

MapReduce Phases

Map Phase

Each mapper processes one block.

Outputs key-value pairs.

Example:

(word, 1)

Shuffle & Sort

Framework:

- Groups same keys
- Routes to same reducer
- Sorts keys

Reduce Phase

Reducer aggregates values.

Example:

(word → total count)

Output stored back in HDFS.

MapReduce and SQL Systems

How SQL Translates

GROUP BY:

- Map → emit (key, 1)
- Shuffle → group keys
- Reduce → sum

Hive compiles SQL into MapReduce jobs.

Limitations of MapReduce

Limitation	Impact
Batch only	High latency
Disk-heavy	Slower pipelines
Single shuffle	Complex queries need chains
No iteration	Poor for ML

Led to rise of Spark.

Conclusion

Key Insights

- HDFS separates metadata and data for scalability
- Large blocks reduce disk seeks and improve throughput
- FsImage and EditLogs maintain metadata reliability
- Checkpointing keeps NameNode efficient
- HA removes single point failures
- Write pipeline ensures parallel replication
- Data locality is central to Hadoop performance
- Rack awareness balances performance and safety
- MapReduce enables large-scale distributed analytics
- SQL engines rely on MapReduce principles

Key Terms

- **NameNode** : Metadata master of HDFS
- **DataNode** : Block storage worker
- **FsImage** : Metadata snapshot
- **EditLog** : Metadata change log
- **Checkpointing** : Merging logs into snapshot
- **Replication Factor** : Number of block copies
- **Rack Awareness** : Topology-based replica placement
- **Data Locality** : Processing near storage
- **MapReduce** : Distributed compute model
- **Mapper** : Processes input blocks
- **Reducer** : Aggregates results
- **Shuffle** : Key grouping stage
- **JournalNodes** : Shared log quorum
- **High Availability** : Active-standby masters